9 – Exercises on Shell, Compilation & Build Systems

Université Jean Monnet – Saint-Étienne 2026/2027

Quick reference

Command line and paths

Command	Effect
`pwd`	Print the current working directory (absolute path)
`ls`	List files and directories in the current directory
`ls <path>`	List files and directories at `<path>`
`cd <path>`	Change the current directory to `<path>`
`cd ..`	Go up one level (parent directory)
`cd ~`	Go to your home directory

An **absolute path** starts from the root of the filesystem: `/home/alice/music/tracks`.

A **relative path** starts from the current directory: `music/tracks` or `../images`.

Compiling with `clang++`

Command	Effect
`clang++ file.cpp`	Compile `file.cpp`; output binary is `a.out`
`clang++ -o name file.cpp`	Compile and name the binary `name`
`clang++ -o name a.cpp b.cpp`	Compile multiple source files into one binary
`clang++ -I<dir>`	Add `<dir>` to the header search path
`./name`	Run the binary

Linking against libraries

Flag		Meaning
`-L<dir>`		Tell the linker to look for libraries in `<dir>`
`-l<name>`		Link against `lib<name>.so` (or `<name>.dll/lib`, or `<name>.dylib`)
`LD_LIBRARY_PATH`	(Linux)	Colon-separated list of directories searched at runtime for
`DYLD_LIBRARY_PATH`	(MacOS)	shared libraries
`PATH` (Windows)		

Example: a library `libsndfile.so` sits in `/usr/local/lib` and its headers are in `/usr/local/include`.

```
clang++ -o player main.cpp -I/usr/local/include -L/usr/local/lib -
    lsndfile
```

If the library is a *shared* library (`.so`), the OS must also be able to find it at *runtime*:

```
export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH
./player
```

Building with CMake

Command	Effect
`cmake -B <build_dir>`	Configure the project; create `<build_dir>`
`cmake -B <build_dir> -S <src_dir>`	Configure, specifying where `CMakeLists.txt` is
`cmake --build <build_dir>`	Compile and link everything
`cmake --build <build_dir> --target <name>`	Build only the named target

A typical workflow:

```
cd my_project           # CMakeLists.txt is here
cmake -B build          # configure; creates my_project/build/
cmake --build build     # compile; binary appears inside build/
./build/my_binary       # run
```

Problem 1 — Paths, navigation and basic shell commands

1.1 — What does `pwd` print?

The filesystem looks like this. Alice's current directory is `/home/alice/projects/sound`.

```
home/
└── alice
    ├── projects/
    │   ├── sound/
    │   │   ├── main.cpp
    │   │   └── filters
    │   │       └── reverb.cpp
    │   └── images/
    │       ├── main.cpp
    │       └── data
    │           └── photo.png
    └── downloads/
        └── lib.zip
```

After each command sequence below, write what `pwd` would print.

Each row is independent: always start from `/home/alice/projects/sound`.

Commands	`pwd` output
(starting position, no command)	
`cd ..`	
`cd ../images`	
`cd filters`	
`cd ../../downloads`	
`cd /home/alice`	

1.2 — Absolute or relative?

Classify each path as **absolute** or **relative**.

Path	Absolute or relative?
`/home/alice/projects`	
`../images/data`	
`filters/reverb.cpp`	
`/usr/lib`	
`./main.cpp`	

1.3 — Fill in the command

Download this archive and unzip it somewhere on your computer. Open a terminal in the folder `projects` that the archive contains. Write a single shell command that achieves each goal. (Each path in the following table is relative to the folder `alice/`).

Goal	Command
List the contents of `projects/sound/filters`	
Go to `projects/images/data`	
List the contents of `downloads` without leaving the current directory	
Go to the home directory	

Problem 2 — Compiling with `clang++`

2.1 — Compile by hand

Open the folder `/home/alice/projects/sound` from the archive of problem 1.3. It contains a single file `main.cpp` and a folder `filters`.

In the VSCode terminal, copy-paste and execute the following commands:

Command	Goal
`clang++ -std=c++17 main.cpp filters/reverb.cpp`	Compile `main.cpp` with the default output name
`clang++ -std=c++17 main.cpp filters/reverb.cpp -o sound`	Compile the program and name the binary `sound`
`./sound`	Run the resulting binary `sound`

Note: if you are on Windows, your compiler is actually named `clang++.exe`.

2.2 — Multiple files and headers

Download this archive and unzip it somewhere on your computer. Open a terminal in the folder `alice/projects/sound`.

`main.cpp` includes `include/filters.hpp` with:

```
#include <filters.hpp>
```

The function `reverb` used in `main` is defined in `filters/reverb.cpp`.

In your terminal, copy-paste and execute the following command:

Goal	Command
Compile the project into a binary named `sound`	`clang++ -std=c++17 main.cpp filters/reverb.cpp -o sound -I include`

Note: if you are on Windows, your compiler is actually named `clang++.exe`.

Problem 3 — Linking against libraries

3.1 — Write the command

Download this archive, unzip it somewhere you can access it. Open a terminal and navigate to the folder `alice/projects/sound`. The project layout is:

```
main.cpp
libs/
    include/
        reverb.h
    lib/
        macos/
            libreverb.dylib
        linux/
            libreverb.so
        windows/
            reverb.dll
            reverb.lib
```

In your terminal, copy-paste and execute the commands from table 1.

Table 1: Commands to execute

macOS	Windows (MSVC)	Goal
`clang++ -std=c++20 main.cpp -Ilibs/include -o sound -Llibs/lib/macos/ -lreverb`	`clang++.exe main.cpp -Ilibs\include -o sound.exe -Llibs\lib\windows -lreverb`	Compile `main.cpp` into `sound`, with headers and the `reverb` library
`export DYLD_LIBRARY_PATH=$PWD/libs/lib/macos:$DYLD_LIBRARY_PATH`	`set PATH=%CD%\libs\lib\windows;%PATH%`	Set your environment so the OS finds the reverb library at runtime
`./sound`	`sound.exe`	Run the binary

Problem 4 — Building with CMake

4.1 — Fill in the command

Download this archive and unzip it somewhere on your computer. Open a terminal in the folder `alice/projects/sound`.

```
CMakeLists.txt
main.cpp
filters/
    reverb.cpp
include/
    reverb.hpp
```

In your terminal, copy-paste and execute the following commands:

Command	Goal
`cmake -S . -B build`	Configure the project into a folder called `build`
`cmake --build build`	Compile the project
`cmake --build build`	Rebuild after alterting `filters.cpp`, e.g add a `std::cout` somewhere (same command)